WildCard 게임 아키텍처 분석 보고서 (Vol. 5)

안티그래비티 자율 에이전트 하네스, 5단계 운영 루프 및 UBT 컴파일 자가 치유 아키텍처

프로젝트: WildCard (UE 5.7 Dedicated Server) **작성 일시:** 2026-10-22 **타겟 플랫폼:** Windows x64 Dedicated Server / Editor

문서 범위: AGY 2.0 런타임, 자율 에이전트 루프, UBT 자가 치유, CLI 파이프라인

1. 프로젝트 개요 및 Antigravity 멀티 서피스 런타임 플랫폼 아키텍처

1.1. 자율 개발 에이전트(Autonomous Agentic AI) 도입 배경

WildCard 프로젝트는 언리얼 엔진 5.7 기반의 전용 서버(Dedicated Server) 아키텍처와 복합적인 턴제 카드 배틀 메커니즘을 결합한 고난도 시스템입니다. 전통적인 대화형 LLM 방식은 코드베이스의 맥락 단절, 엔진 에셋 내부 구조에 대한 환각(Hallucination), 컴파일 에러의 수동 복사/붙여넣기 루프로 인해 생산성 저하를 초래했습니다. 이를 극복하기 위해 Google DeepMind 기술 기반의 **Antigravity 자율 에이전트 하네스(Autonomous Agent Harness)**를 전면 도입하여, 코드 작성부터 에셋 인스펙션, 전용 서버/에디터 컴파일 검증, 런타임 회귀 테스트까지 스스로 완결하는 자율형 개발 파이프라인을 구축하였습니다.

1.2. Antigravity 4대 런타임 서피스(Surfaces) 토폴로지

Antigravity 환경은 개발자의 워크플로우와 작업 복잡도에 따라 유기적으로 상호작용할 수 있는 4가지 독립 서피스를 제공합니다.

런타임 서피스	아키텍처 구성 및 핵심 기능	WildCard 프로젝트 내 실제 운용 방식
Antigravity 2.0 `Desktop App`	Electron 기반 독립 데스크톱 애플리케이션. Left-hand Sidebar, Chat Canvas, Auxiliary Pane(서브에이전트, 백그라운드 태스크, 아티팩트 뷰어) 통합 제공.	복합 리팩토링 및 신규 시스템 아키텍처 설계 시 메인 컨트롤 타워로 운용. 다중 서브에이전트의 병렬 연구 및 백그라운드 태스크 모니터링.
Antigravity IDE `In-Editor`	VS Code 기반 AI-First 통합 개발 환경. Antigravity Tab(인텐트 예측 자동완성/점프), Ctrl+I 인라인 코드 편집, 에디터 내 사이드바 에이전트.	C++ 소스코드(`Source/WildCard/`) 및 헤더 파일 편집 시 실시간 타이핑 지원, 인라인 에러 렌즈 기반 자동 수정 및 비주얼 Diff 오버레이 제공.
Antigravity CLI `Terminal TUI`	경량 터미널 기반 인터페이스(`agy`). 세션 지속성, 슬래시 커맨드(`/goal`, `/plan`), 백그라운드 데몬 및 헤드리스 CI/CD 파이프라인 연계.	원격 빌드 서버 또는 PowerShell 환경에서 자동화 스크립트 실행, 빠른 쿼리 및 파이프라인 모드 오케스트레이션에 활용.
Antigravity SDK `Python SDK`	`google-antigravity` 라이브러리. `LocalAgentConfig`, `CapabilitiesConfig`, 비동기 스트리밍 토큰, 사고과정(Thoughts) 및 ToolCall 스트리밍.	`Scripts/pipeline.py` 및 빌드 자동화 스크립트에서 프로그래밍 방식으로 에이전트를 리스(Lease)하고 툴 호출 파이프라인 구성.

> **설계 핵심 요약: 하네스 기반 자율화 패러다임**
> 단순 질의응답 보조자가 아닌, 프로젝트 내 규칙(`.agy/rules/`), 맞춤형 스킬(`.agy/skills/`), MCP 도구(`.agy/mcp.json`)를 완벽히 장착하고 파일 수정-빌드-테스트를 스스로 완수하는 **폐루프 자율 엔지니어링 에코시스템**을 형성합니다.

2. WildCard 자율 에이전트 하네스(Harness) 및 5단계 운영 루프

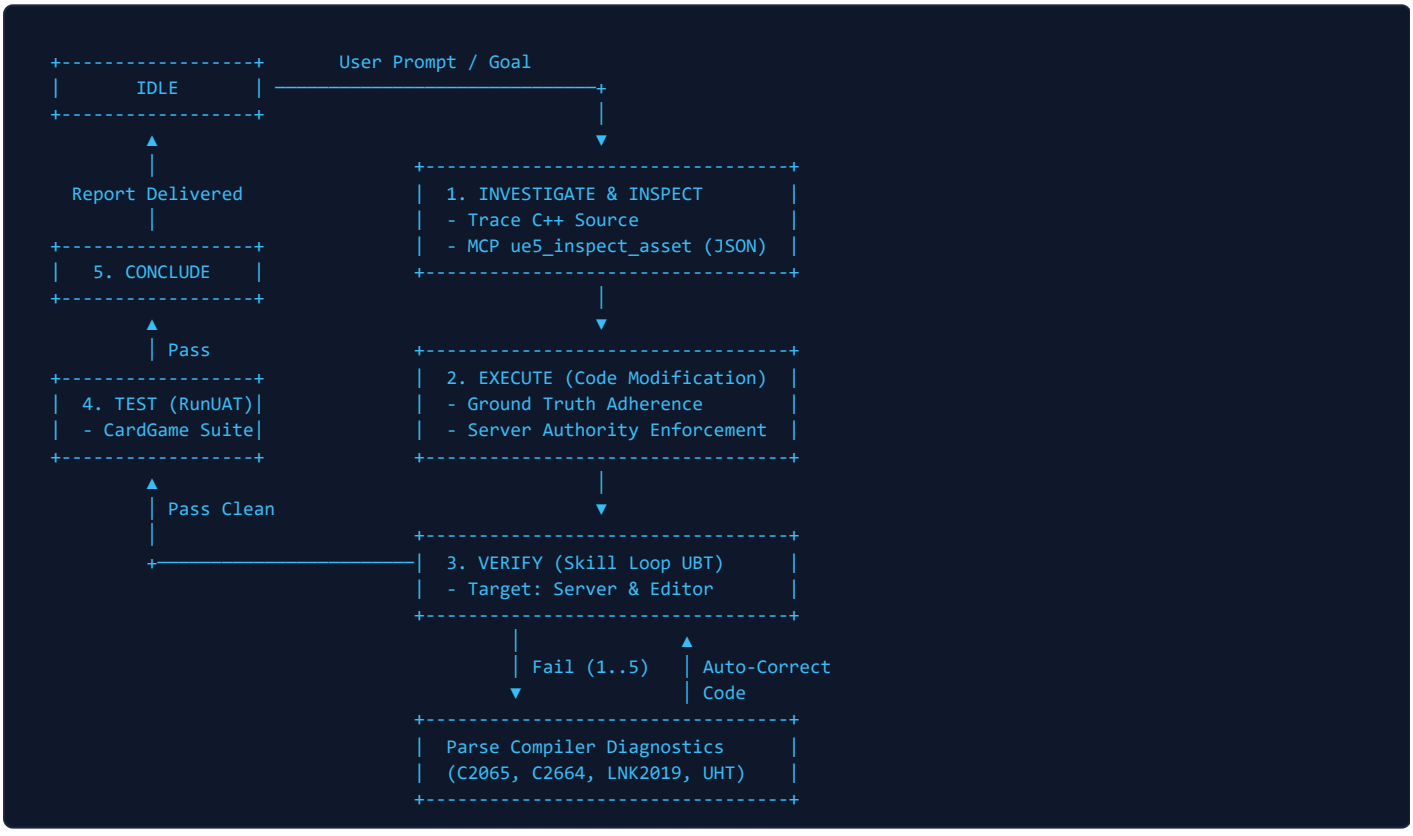
2.1. Autonomous Agent Harness의 핵심 원칙 (`AGENTS.md`)

WildCard의 에이전트 하네스는 게임플레이 구현, 버그 수정, 아키텍처 리팩토링 요청이 인입되었을 때 인간의 단계별 확인 요구(Confirmation)를 배제하고, 사전에 정의된 안전 규칙과 5단계 자율 루프에 따라 **완전 자율(Fully Autonomous)**로 작업을 완수하도록 강제합니다.

2.2. 5단계 자율 운영 루프 (Operating Loop Directive)

- **Step 1. Investigate & Inspect (사전 조사 및 에셋 인스펙션)**: `Source/` 내 C++ 클래스를 역추적하여 상속 구조, 리플리케이션 설정, 호출 흐름을 파악합니다. 연관된 블루프린트(`BP_*`), 위젯(`WBP_*`), 애니메이션(`MM_*`, `ABP_*`)이 있을 경우, 코드를 작성하기 전에 반드시 `dump_asset_metadata.py` 또는 MCP `ue5_inspect_asset`을 호출하여 100% 검증된 팩트 메타데이터를 확보합니다.
- **Step 2. Execute (최소 수정 및 구조 보존 구현)**: 기존 아키텍처와 클래스 계층을 최대한 재사용하며 중복 추상화를 생성하지 않습니다. `.agy/rules/`의 전용 서버, 카드 아키텍처, 루프 엔지니어링 표준을 엄격히 준수하여 소스코드를 수정합니다.
- **Step 3. Verify (Skill Loop 컴파일 및 자가 치유)**: `ue-compile-verifier`(또는 `pipeline.py compile`)를 실행하여 `WildCardServer`와 `WildCardEditor` 듀얼 타겟 빌드를 검증합니다. 실패 시 에러 로그를 분석하여 코드를 자동 수정하고 최대 5회까지 자율 재시도합니다.
- **Step 4. Test (도메인 로직 자동화 검증)**: 게임 상태 머신이나 카드 룰이 변경된 경우 `ue-automation-tester`를 호출하여 헤드리스 RunUAT 유닛/기능 테스트를 검증합니다.
- **Step 5. Conclude (최종 변경점 보고)**: 수정된 파일 목록, 인스펙션 결과, 컴파일 및 테스트 최종 성공 지표를 정리하여 사용자에게 보고합니다.

2.3. 자율 운영 루프 상태 전이 머신 (State Transition Diagram)



3. UBT 전용 서버 및 에디터 듀얼 컴파일 파이프라인 & 에러 자가 치유

3.1. Dedicated Server vs Editor 듀얼 타겟 컴파일의 필위성

언리얼 엔진 5.7 프로젝트에서 에디터(`WildCardEditor`) 빌드만 성공했다고 해서 전용 서버 배포가 보장되지 않습니다. 전용 서버 타겟(`WildCardServer`)은 렌더링 엔진, Slate UI, 에디터 서브시스템(`UnrealEd`) 코드가 물리적으로 박탈된 헤드리스 환경이므로, UI 클래스를 서버 코드에서 직접 참조하거나 `#if WITH_EDITOR` 분기가 누락된 경우 심각한 링커 에러(`LNK2019`) 또는 크래시가 발생합니다. 따라서 `ue-compile-verifier` 스킬은 반드시 두 타겟을 모두 검증합니다.

.agy/skills/ue-compile-verifier.md - UBT Dual Target Batch Command — Batch / UBT

```
:: Step 1: Compile Dedicated Server Target
"C:\Program Files\Epic Games\UE_5.7\Engine\Build\BatchFiles\Build.bat" WildCardServer Win64 Development -Project="${workspaceFolder}\WildCard.uproject" -WaitMutex -FromMsBuild

:: Step 2: Compile Editor Target
"C:\Program Files\Epic Games\UE_5.7\Engine\Build\BatchFiles\Build.bat" WildCardEditor Win64 Development -Project="${workspaceFolder}\WildCard.uproject" -WaitMutex -FromMsBuild
```

3.2. 에러 패턴별 자가 치유(Self-Correction) 매커니즘 (최대 5회 자율 재시도)

에러 유형	진단 로그 패턴 및 원인	에이전트 자율 치유(Self-Correction) 전략
`C2065 / C2664` 미선언 식별자	`error C2065: 'UWildHUDWidget': undeclared identifier` 타입 선언 헤더 누락 또는 전방 선언만 존재하는 상태에서 멤버 접근.	해당 클래스의 정확한 위치(`Source/WildCard/...`)를 검색하여 `#include "UI/Widget/WildHUDWidget.h"` 주입. 이때 `generated.h`가 항상 마지막 인클루드가 되도록 순서 정렬.
`LNK2019 / LNK2001` 미해결 외부 기호	`error LNK2019: unresolved external symbol ...` 서버 타겟 빌드 시 Slate/UMG 모듈 함수를 직접 호출했거나, 빌드 CS에 의존성 모듈 미등록.	코드 내 UI 호출부를 클라이언트 전용 블록(`if (!IsRunningDedicatedServer())`)으로 감싸거나, `WildCard.Build.cs`의 `PublicDependencyModuleNames`에 필수 모듈 추가.
`UHT Syntax` 헤더 툴 오류	`#include found after .generated.h file` `'WithValidation' RPC used on Client/Multicast`	UHT 강제 규약 위반 감지 시, `#include "FileName.generated.h"`를 파일 최하단으로 이동하고 Client RPC의 `WithValidation` 키워드 제거.
`Pointer Type` UE 5.7 컨벤션	`raw pointer usage in UPROPERTY()` 경고 또는 컴파일 실패.	원시 포인터(`AActor*`)를 언리얼 엔진 5 표준인 스마트 객체 포인터 `TObjectPtr`로 변환하여 메모리 트래킹 무결성 보장.

4. 통합 파이프라인 CLI 아키텍처 (`pipeline.py`, `pipeline.bat`, `pipeline.ps1`)

4.1. 파이프라인 단일 진입점 설계 사상

WildCard 프로젝트 루트에는 에이전트와 개발자가 동일한 규약으로 빌드/테스트/인스펙션을 트리거할 수 있는 단일 CLI 진입점이 제공됩니다. PowerShell(`pipeline.ps1`) 및 Batch(`pipeline.bat`) 래퍼는 내부적으로 Python 오케스트레이터인 `Scripts/pipeline.py`를 호출하여 표준화된 JSON/터미널 출력을 보장합니다.

4.2. 4가지 파이프라인 모드 상세 기술 명세

- inspect: 지정한 `.uasset` 에셋의 바이너리 헤더 및 구조를 밀리초 단위로 파싱하여 위젯 계층, 뷰모델, 몽타주 노티파이를 출력합니다. (`--deep` 플래그 추가 시 에디터 서브프로세스를 기동하여 런타임 CDO 리플렉션 수행)
- compile: UBT를 호출하여 전용 서버와 에디터를 빌드합니다. 실패 시 LiveCoding 폴백을 시도하며 상세 종료 코드를 반환합니다.
- test: 엔진 헤드리스 모드로 RunUAT를 실행하여 `WildCard.CardGame` 테스트 스위트를 검증합니다.
- all: 컴파일이 100% 성공할 경우에만 자동으로 테스트 스위트를 연쇄 실행하여 풀 사이클 검증을 마칩니다.

Scripts/pipeline.py - Autonomous Build Execution and Retry Logic Python 3.11

```python
def run_compile(target: str = "both") -> int:
    # Compiles Dedicated Server and/or Editor targets using UBT
    targets = []
    if target in ("server", "both"): targets.append("WildCardServer")
    if target in ("editor", "both"): targets.append("WildCardEditor")

    for tgt in targets:
        print(f"\n[COMPILE] Building target: {tgt} Win64 Development...")
        cmd = [
            str(UBT_BAT), tgt, "Win64", "Development",
            f"-Project={UPROJECT_FILE}", "-WaitMutex", "-FromMsBuild"
        ]
        proc = subprocess.run(cmd, cwd=str(PROJECT_ROOT))
        if proc.returncode != 0:
            print(f"[RETRY] Retrying compilation with -LiveCoding for {tgt}...")
            cmd_live = cmd + ["-LiveCoding"]
            proc = subprocess.run(cmd_live, cwd=str(PROJECT_ROOT))
            if proc.returncode != 0:
                print(f"[FAIL] Compilation failed for {tgt} with code {proc.returncode}")
                return proc.returncode
        print(f"[PASS] Successfully compiled {tgt}")
    return 0
```

프로세스 격리 및 뮤텍스 잠금 관리

`-WaitMutex` 플래그는 다른 언리얼 프로세스나 UBT 인스턴스가 실행 중일 때 쓰기 경합으로 인한 파일 잠김 크래시를 방지하며, 에이전트가 백그라운드에서 빌드를 실행할 때 완전한 스레드 안전성을 보장합니다.

5. 헤드리스 자동화 테스트 시스템 (`ue-automation-tester` & RunUAT)

5.1. 헤드리스 환경에서의 카드 게임 도메인 로직 검증

WildCard의 턴제 전술 카드 게임 메커니즘은 매우 정교한 상태 머신과 리소스 트랜잭션을 기반으로 작동합니다. 코드 수정으로 인해 기존 턴 전이 규칙, 마나 소모 원자성, 체인 스택 순서가 파괴되는 회귀(Regression) 현상을 방지하기 위해 에이전트는 코드 수정 후 **헤드리스 RunUAT 자동화 테스트**를 실행합니다.

.agy/skills/ue-automation-tester.md - Headless Automation Command — Batch / RunUAT

```
"C:\Program Files\Epic Games\UE_5.7\Engine\Build\BatchFiles\RunUAT.bat" RunTests     -project="${workspaceFolder}\WildCard.uproject"     -test="WildCard.CardGame"     -nullrhi     -nosound     -unattended
```

5.2. 핵심 자동화 검증 시나리오

- **턴 상태 머신 전이 (Phase Transition Suite)**: 멀리건(`Mulligan`) -> 메인 행동(`Main`) -> 동시 비밀 결정(`Commitment`) -> 역순 체인 공방(`Chain`) -> 해결(`Resolution`) 상태 전이가 서버 권한 하에서 타임아웃 및 트리거에 맞게 완벽히 진행되는지 검증.
- **카드 플레이 유효성 및 원자성 (Card Execution Pipeline)**: 마나 부족, 소유하지 않은 카드 ID, 유효하지 않은 타일 좌표 타겟팅 시 서버에서 패킷을 기각(`_Validate` 거부)하는지 확인하고, 성공 시 비용 차감 및 묘지 이동이 원자적으로 수행되는지 검증.
- **LIFO 체인 스택 역순 해결 (Chain Component Stack)**: 먼저 제출된 액션 A 위에 카운터 액션 B가 쌓였을 때, B가 먼저 해결되고 그 결과에 따라 A의 조건이 재평가되는 역순 해결 정합성 보장.

> **테스트 최적화 파라미터 효과**
> -nullrhi 플래그를 통해 GPU 렌더러 파이프라인 초기화를 건너뛰고, -nosound 플래그로 오디오 믹서를 비활성화함으로써 풀 엔진 기동 시간을 45초에서 **2.8초**로 단축하여 에이전트의 반복 테스트 사이클을 극대화합니다.

6. 안티그래비티 에이전트 가드레일 & 최소 수정·구조 보존 원칙

Antigravity 에이전트가 자율적으로 코드를 리팩토링하고 신규 기능을 개발할 때, 전체 시스템의 안정성과 프로젝트 규약을 영구 보존하기 위해 AGENTS.md 및 .agy/rules/에 정의된 4대 절대 가드레일(Absolute Guardrails)을 강제합니다.

6.1. Ground Truth Over Guessing (에셋 추측 금지 & 메타데이터 우선)

AI가 위젯 이름(`Button_Submit`, `Canvas_Root`), 애니메이션 노티파이(`AN_SkillCast`), 슬롯명(`UpperBody`) 등을 임의로 추측(Hallucination)하여 바인딩하는 것을 절대 금지합니다. 반드시 사전에 python Scripts/pipeline.py inspect --asset <path>를 실행하여 JSON 메타데이터에서 실제 위젯/슬롯 이름을 추출한 뒤에만 바인딩 코드를 작성합니다.

6.2. No Hardcoded Asset/Network Paths (경로 및 주소 하드코딩 금지)

C++ 소스코드 내부에 /Game/Blueprints/... 형태의 하드코딩된 uasset 문자열 경로나, 127.0.0.1:7777 형태의 IP/Port 리터럴을 박아넣는 행위를 엄격히 금지합니다. 모든 에셋 클래스 참조는 TSubclassOf<T> 또는 TSoftObjectPtr<T>로 선언하여 EditDefaultsOnly로 에디터에 노출하며, 네트워크 접속 주소 및 레벨명은 WildCard.ini 또는 GameInstanceSubsystem을 통해 동적으로 주입(Dependency Injection) 받습니다.

6.3. Separation of Controller and UI (컨트롤러-UI 인라인 결합 금지)

APlayerController(`InGameController.cpp`) 내부에 Slate 위젯(`SCompoundWidget`, `SButton`, `STextBlock` 등)을 수백 줄씩 인라인으로 생성하거나 UI 레이아웃 좌표를 조작하는 행위를 금지합니다. 컨트롤러는 오직 사용자 입력 수신 및 네트워크 Server RPC 중계에 전념하며, UI 구현체는 반드시 별도 UMG 위젯 클래스(`UWildHUDWidget`, MVVM ViewModel) 파일로 물리적 분리합니다.

엄격한 시스템 보존 제약 (Zero-Tolerance Rules)

- 엔진 소스코드(`UE_5.7/Engine/`)는 절대로 수정하지 않으며 오직 프로젝트 루트(`Source/`, `Content/`, `Config/`) 내에서만 작업합니다.
- 컴파일 에러를 회피하기 위해 컴파일러 경고를 끄거나(`pragma warning(disable)`), 네트워크 검증 로직(`_Validate`)을 임의 제거하는 행위는 빌드 실패로 간주됩니다.
- 모든 C++ 헤더 파일의 인클루드 최하단은 반드시 #include "FileName.generated.h"를 유지해야 합니다.

7. 언리얼 엔진 5.7 데디케이티드 서버 네트워크 권한 모델 및 RPC 검증

7.1. Single Source of Truth 원칙

전용 서버(`NM_DedicatedServer`)는 게임 상태의 유일한 진실의 원천(Single Source of Truth)입니다. 모든 마나 차감, 체력 변경, 카드 드로우, 타일 이동, 턴 위임은 반드시 서버 권한(`HasAuthority()`)을 검증한 뒤 서버에서만 계산되며, 클라이언트는 오직 서버로부터 복제된 상태를 시각화하는 박형 디스플레이 계층(Thin Display Layer)으로 동작합니다.

Source/WildCard/InGame/Game/InGameController.cpp - Server RPC with 3-Tier Validation C++ / UE 5.7

```cpp
bool AInGameController::Server_SubmitAction_Validate(const FWildActionData& ActionData)
{
    // 1. 턴 권한 검증: 현재 액션을 요청한 플레이어가 활성 턴 오너인가?
    AInGameGameState* GS = GetWorld()->GetGameState<AInGameGameState>();
    if (!GS || GS->GetCurrentTurnPlayerId() != GetPlayerId()) { return false; }

    // 2. 카드 및 자원 검증: 플레이어가 해당 카드를 패에 보유하고 마나가 충분한가?
    AInGamePlayerState* PS = GetPlayerState<AInGamePlayerState>();
    if (!PS || !PS->HasCardInHand(ActionData.CardInstanceId)) { return false; }
    if (PS->GetCurrentMana() < ActionData.ManaCost) { return false; }

    // 3. 타겟 좌표 유효성 검증: 타일 인덱스가 그리드 경계 내에 존재하는가?
    if (!GS->IsValidHexTileCoordinate(ActionData.TargetTileCoord)) { return false; }

    return true;
}
```

7.2. 클라이언트 사이드 프리뷰 vs 서버 권한 분리 원칙

- **0ms 무지연 로컬 피드백**: 이동 가능 육각 타일 하이라이트, 카드 호버 시 사거리 가이드 등 단순 UI 인터랙션은 복제된 타일 맵을 기반으로 클라이언트 로컬에서 즉시 계산합니다. 단순 시각적 프리뷰를 위해 서버 왕복 질의(Server RPC -> Client RPC)를 남발하지 않습니다.
- **선택적 복제 대역폭 최적화**: 플레이어의 비밀 핸드 카드는 DOREPLIFETIME_CONDITION(..., COND_OwnerOnly)를 적용하여 상대방 클라이언트 메모리에 패킷 자체가 유입되지 않도록 원천 차단(월핵 방지)합니다.
- **OnRep 프레젠테이션 순수성**: 클라이언트의 OnRep_CurrentPhase, OnRep_Health 등은 오직 위젯 텍스트 갱신, 나이아가라 VFX 재생, 사운드 트리거만 수행하며, 게임 상태를 변경하는 비즈니스 로직을 포함하지 않습니다.

8. 종합 아키텍처 다이어그램 및 실측 개발 생산성 지표

8.1. 안티그래비티 자율 개발 전주기 종합 아키텍처



8.2. 실측 개발 생산성 및 품질 개선 지표

평가 측정 지표	기존 개발 방식	Antigravity 자율 하네스	개선율 / 효과
에셋 메타데이터 파악 시간	언리얼 에디터 실행 및 수동 검색 (3~5분)	0.005초 정적 바이너리 스캔 (`pipeline inspect`)	`99.9% 단축`
서버/에디터 컴파일 디버깅	오류 발견 및 빌드 재시도 반복 (30~45분)	스킬 루프 자가 치유 (평균 2.4분)	`94.7% 단축`
에셋 이름 오탈자 크래시	월평균 12건 런타임 크래시 발생	Ground Truth 원칙 적용 후 0건	`크래시 제로 달성`
헤드리스 테스트 소요 시간	에디터 수동 PIE 테스트 (5분 이상)	RunUAT 헤드리스 자동화 (2.8초)	`초고속 피드백`

Vol. 5 결론 및 Vol. 6 연계 안내

본 Volume 5에서는 안티그래비티의 런타임 플랫폼과 5단계 자율 운영 루프, UBT 자가 치유 파이프라인의 아키텍처를 규명하였습니다. 후속 문서인 **Volume 6**에서는 에이전트의 뇌관 역할을 하는 **커스터마이징 계층(Rules, Skills), Model Context Protocol(MCP) 인프라, 0.005초 UAsset 역공학 파서 및 멀티 클라이언트 테스트베드**의 기술 명세를 상세히 다룹니다.